S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

Practical No. 07

Aim: Write a Prolog Program to demonstrate the concept of Family relationship.

Name of Student:

Roll No.: Semester/Year: VI/III

Academic Session:

Date of Performance:

Date of Submission:

AIM: Write a Prolog Program to demonstrate the concept of Family relationship.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- Represent knowledge using **facts**
- Define rules using **logical inference**
- Understand **unification**
- Observe **backtracking**
- Model **hierarchical relationships** (parent, sibling, grandparent, etc.)

THEORY:

Prolog (Programming in Logic) is a **declarative logic programming language** based on:

- First-Order Predicate Logic
- Resolution
- Unification
- Backtracking

Unlike procedural languages (C, Python), Prolog focuses on **what is true**, not **how to compute it**.
Common implementation: **SWI-Prolog**
In Prolog a program consists of:

1. Facts
2. Rules
3. Queries

Installing Prolog (Using SWI-Prolog)

Step 1: Download

Visit: https://www.swi-prolog.org

Step 2: Install

Follow OS-specific installer.

Step 3: Run

Open terminal / command prompt:

swipl

?- This is the Prolog query prompt.

Prolog data types:

- **Atoms** → `john`, `apple`
- **Variables** → `X`, `Person`
- **Numbers** → `10`, `3.14`
- **Structures** → `parent(john, mary)`
- **Lists** → `[1,2,3]`

Unification:

Prolog matches patterns for unification. Exa:

parent (john, X) Unifies with: parent (john, mary)

Result:

```
X = mary
```

Backtracking:

Prolog automatically searches for alternatives using backtracking.
Example Query:

?- parent (john, X).

Press `;` to get next solution.

Important Operators:

Operator	Meaning
`:-`	If
`,`	AND
`;`	OR
`=`	Unification
`is`	Arithmetic evaluation
`\=`	Not unifiable
`\+`	Negation as failure

A **family relationship system** is a classical example used in Artificial Intelligence to demonstrate:

- Knowledge representation
- Predicate logic
- Recursive reasoning
- Depth-first search with backtracking

ALGORITHM:

1. Start
2. Define family members using `male/1` and `female/1` facts
3. Define parent elationships
4. Define derived relationships using rules:

    a. father
    b. mother
    c. sibling
    d. grandparent
    e. ancestor

5. Load the program in Prolog
6. Execute queries
7. Observe results and backtracking
8. Stop

9. Define derived relationships using rules:

    a. father
    b. mother
    c. sibling
    d. grandparent
    e. ancestor

10. Load the program in Prolog
11. Execute queries
12. Observe results and backtracking
13. Stop

PROGRAM CODE:

```
% -------- PARENT FACTS (ALL TOGETHER) --------
parent(shankar, kishor). parent(laxmi, kishor).
parent(kishor, vedant). parent(anita, vedant).
parent(vedant, rohan). parent(sneha, rohan).
parent(vedant, priya). parent(sneha, priya).
% -------- MALE FACTS (ALL TOGETHER) --------
male(shankar). male(kishor). male(vedant). male(rohan).
% -------- FEMALE FACTS (ALL TOGETHER) --------
female(laxmi). female(anita). female(sneha). female(priya).
% -------- MARRIAGE FACTS (GROUPED) --------
husband(shankar, laxmi).
husband(kishor, anita).
husband(vedant, sneha).
wife(laxmi, shankar).
wife(anita, kishor).
wife(sneha, vedant).  father(X, Y) :-
parent(X, Y),
male(X).
mother(X, Y) :-
parent(X, Y),
female(X).
child(X, Y) :-
parent(Y, X).
son(X, Y) :-
child(X, Y),
```

```
male(X).

daughter(X, Y) :-

child(X, Y),

female(X).

sibling(X, Y) :-

parent(Z, X),

parent(Z, Y),

X \= Y.

grandparent(X, Y) :-

parent(X, Z),

parent(Z, Y).

great_grandparent(X, Y) :-

parent(X, A),

parent(A, B),

parent(B, Y).

grandfather(X, Y) :-

grandparent(X, Y),

male(X).

grandmother(X, Y) :-

grandparent(X, Y),

female(X).
```

INPUT & OUTPUT:

Step 1: Start Prolog

```
swipl
```

Step 2: Consult File

```
?- consult('family.pl').
```

OR

```
?- [family].
```

Queries & Output

SWISH File Edit Examples Help 191 users online Search

Program

```
% -------- PARENT FACTS --------

parent(grandfather, kishor).
parent(grandmother, kishor).

parent(kishor, vedant).
parent(anita, vedant).

parent(vedant, aryan).
parent(sneha, aryan).

% -------- MALE FACTS --------

male(grandfather).
male(kishor).
male(vedant).
male(aryan).

% -------- FEMALE FACTS --------

female(grandmother).
female(anita).
female(sneha).

% -------- MARRIAGE FACTS --------

husband(grandfather, grandmother).
husband(kishor, anita).
husband(vedant, sneha).

wife(grandmother, grandfather).
wife(anita, kishor).
```

father(X, vedant)
X
kishor 1

mother(X, vedant)
X
anita 1

son(vedant, kishor).
true 1

child(vedant, X).
X
kishor 1
Next 10 100 1,000 Stop

son(vedant, kishor).
true 1

child(vedant, X).
X
kishor 1
Next 10 100 1,000 Stop

parent(X, vedant).
X
kishor 1
Next 10 100 1,000 Stop

?- Examples History Solutions table results Run!

SWISH File Edit Examples Help 193 users online Search

Program

```
female(grandmother).
female(anita).
female(sneha).

% -------- MARRIAGE FACTS --------

husband(grandfather, grandmother).
husband(kishor, anita).
husband(vedant, sneha).

wife(grandmother, grandfather).
wife(anita, kishor).
wife(sneha, vedant).

% -------- RULES --------

father(X, Y) :- parent(X, Y), male(X).
mother(X, Y) :- parent(X, Y), female(X).

child(X, Y) :- parent(Y, X).

son(X, Y) :- child(X, Y), male(X).
daughter(X, Y) :- child(X, Y), female(X).

sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.

grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

great_grandparent(X, Y) :- parent(X, A), parent(A, B), parent(B, Y).

grandfather(X, Y) :- grandparent(X, Y), male(X).
grandmother(X, Y) :- grandparent(X, Y), female(X).
```

father(X, vedant)
X
kishor 1

mother(X, vedant)
X
anita 1

son(vedant, kishor).
true 1

child(vedant, X).
X
kishor 1
Next 10 100 1,000 Stop

son(vedant, kishor).
true 1

child(vedant, X).
X
kishor 1
Next 10 100 1,000 Stop

parent(X, vedant).
X
kishor 1
Next 10 100 1,000 Stop

?- Examples History Solutions table results Run!

CONCLUSION:

DISCUSSION QUESTIONS:

1. What is a fact in Prolog?
2. What is the difference between fact and rule?
3. What is unification?
4. Explain backtracking with example.
5. Why is recursion used in `ancestor` rule?

6. What is the difference between = and `is`?

_____

_____

_____

_____

REFERENCES:

- https://www.swi-prolog.org/
- https://www.tutorialspoint.com/prolog/prolog_introduction.htm
- https://www.geeksforgeeks.org/artificial-intelligence/prolog-an-introduction/
- https://www.youtube.com/watch?v=dKn-BbS_zOQ